מעבר ליצירת קוד

סדנה מעשית: פיתוח אג'נטי עם InterSystems IRIS

באמצעות Codex OpenAI + VS Code + שרת MCP iris-agentic-dev

סקירה כללית

מן הסתם אתם כבר משתמשים ב-LLMs ליצירת קטעי קוד - סדנה זו עושה את הצעד הבא: חיבור סוכן בינה מלאכותית ישירות לשרת IRIS חי, כך שהוא עובד בתוך תהליך הפיתוח האמיתי שלכם - מבצע פעולות ומגיב למשוב אמיתי, לא רק מחזיר טקסט.

בסוף הסדנה יהיה לכם:

- הגדרה מלאה ועובדת: Codex agent → MCP server → IRIS.
- הבנה של ההבדל בין Tools ל-Skills.
- קבוצה של דפוסי עבודה בטוחים ומעשיים לשימוש מיידי.

מושג: לולאת ה-Agent

פרומפט יחיד מחזיר תשובה אחת. Agent מריץ מחזור וממשיך עד שהמטרה מושגת או שתנאי עצירה מופעל:

שלב		מה קורה
1.	תכנן	פירוק המטרה לשלבים קונקרטיים.
2.	פעל	קריאה ל-tool — קומפילציה, שאילתה, הרצת פקודה.
3.	התבונן	קריאת הפלט האמיתי והשגיאות ישירות.
4.	תקן	תיקון וניסיון חוזר — אוטומטית.

מה שהופך את זה לאג'נטי: **ה-agent מנהל באופן עצמאי את כל המחזור ללא צורך בהדבקה ידנית בין הצ'אט ל-IDE.**

לולאת הפיתוח ב-IRIS

לכל שלב בתהליך הפיתוח הקיים של IRIS שלכם, יש יכולת agent מתאימה:

פעולה	Tool בשימוש	יכולת ה-Agent
הבן	`iris_doc` `docs_introspect`	חיפוש וסיכום של classes, methods, מסמכים
כתוב / שנה	`Write` `edit`	יצירת ObjectScript עם דפוסים ייחודיים ל-IRIS
קמבל	`iris_compile`	קומפילציה, קריאת שגיאות ישירות, ותיקון אוטומטי
הרץ / שאל	`iris_execute` `iris_query`	הרצת methods ו-SQL, קריאת תוצאות
בדוק	`iris_unit_test`	הרצת בדיקות, בדיקת תוצאות
נפה שגיאות	`iris_execute` `iris_compile`	קריאת שגיאות, מעקב, תיקון

ה-agent יכול לסגור את כל הלולאה הזו ללא יציאה מהצ'אט — זהו ערך הליבה של פיתוח אג'נטי.

ארכיטקטורה: כיצד המרכיבים מתחברים

שכבה	תפקיד	רכיב
לקוח AI	מנפיק קריאות tool בשפה טבעית דרך מפתח ה-API של OpenAI שלכם.	VS Code + OpenAI Codex extension
שרת MCP	מתרגם קריאות tool לבקשות Atelier REST API.	iris-agentic-dev (single Rust binary, runs locally)
IRIS	מבצע קומפילציה, שאילתות, הרצה ומחזיר פלט אמיתי.	Docker container or remote server

ה-Codex agent קורא ל-tool ← שרת ה-MCP מבצע אותו על IRIS פעיל ← שגיאות קומפילציה, הגדרות class ופלט שאילתות מגיעים ישירות חזרה. ה-agent קורא אותם ומתקן בעצמו.

הערה: קיימים שני קבצי תצורה — שמרו עליהם נפרדים:

USERPROFILE%\.codex\config.toml%
רושם את שרת iris-agentic-dev MCP ב-Codex (גלובלי, הגדרה חד-פעמית).

iris-agentic-dev.toml.
מכיל את host, port, namespace ופרטי גישה של IRIS (לפי פרויקט, נמצא בתיקיית ה-workspace של VS Code שלכם).

דרישות מקדימות

וודאו שכל הדרישות הבאות מתקיימות לפני שתתחילו:

דרישה	היכן להשיג	פרטים
VS Code + OpenAI Codex Extension	marketplace.visualstudio.com — search "OpenAI Codex"	לקוח ה-agent. דורש מפתח API של OpenAI.
מפתח API של OpenAI	ינתן בסדנא	משמש את תוסף ה-Codex לקריאות ל-models של OpenAI.
iris-agentic-dev binary	github.com/intersystems-community/iris-agentic-dev/releases/latest	שרת ה-MCP. קובץ Rust בינארי בודד.
Git	git-scm.com	לשכפול ה-repository.
Docker (מומלץ)	docker.com	להרצת IRIS Community מקומית בקונטיינר.

חלק 1 — התקנת iris-agentic-dev

שלב 1: הורדה ומיקום הקובץ הבינארי (יבוצע בהכנות המקדימות לסדנא)

1. צרו תיקייה חדשה `c:\Tools`
2. הוסיפו תיקייה זו למשתנה הסביבה PATH של Windows:
   - לחצו על מקש Windows, הקלידו `environment variables` ולחצו על **Edit the system environment variables**.
   - ב-System Properties, לחצו על **Environment Variables...**
   - תחת **System variables**, גללו ל-**Path**, בחרו אותה ולחצו **Edit**
   - לחצו **New**, הקלידו `C:\Tools`, ולחצו Enter.
   - לחצו OK בכל תיבות הדו-שיח הפתוחות לשמירה.

3. הורידו את **iris-agentic-dev-windows-x86_64.exe** מדף ה-Releases:
   `https://github.com/intersystems-community/iris-agentic-dev/releases/latest`

4. שמרו את הקובץ ב-`c:\Tools` ושנו את שמו ל-`iris-agentic-dev.exe`.
5. פתחו **Command Prompt** **חדש** ואמתו שה-PATH עובד:
   ```
   iris-agentic-dev --version
   ```
   אם מוצגת מספר גרסה, ה-PATH מוגדר כהלכה.

שלב 2: הגדרת תוסף Codex ל-VS Code (יבוצע בסדנא עצמה)

6. רשמו את שרת MCP iris-agentic-dev על ידי העתקת קובץ config.toml למיקום `%USERPROFILE%\.codex\config.toml`:
(תוכן הקובץ מגדיר את החיבוריות ל-MCP: במקום להכניס את הערכים הללו ידנית בהגדרות של ה-Codex Extention).

```
[mcp_servers.iris]
enabled = true
command = "iris-agentic-dev"
args = ["mcp"]

[mcp_servers.iris.env]
IRIS_HOST = "localhost"
IRIS_WEBPORT = "42773"
IRIS_USERNAME = "_SYSTEM"
IRIS_PASSWORD = "SYS"
IRIS_NAMESPACE = "USER"
```

7. בעת הפעילו מחדש את VS Code כדי ש-Codex יקרא את ה-PATH המעודכן — Codex קורא את ה-PATH האתחול.
8. התחברו עם מפתח ה-API של OpenAI בסרגל הפעילות לפתיחת לוח ה-Codex. לחצו על סמל Codex שלכם.

חלק 2 — הגדרת החיבור ל-IRIS

אפשרות א — IRIS ב-Docker (מומלץ)

9. פתחו command line בתיקיית מאגר הסדנה (ai-workshop-day) קיים כבר קובץ בשם compose.yaml. בתיקייה זו והפעילו את IRIS:

```
docker compose up -d
```

10. צרו תיקיית פרויקט VS Code (לדוגמה `c:\VSCode\AIWorkshop`) וצרו קובץ בשם `.iris-agentic-dev.toml` (שימו לב לנקודה בתחילת שם הקובץ) עם התוכן הבא:

```
host      = "localhost"
container = "aiworkshop-iris"
web_port  = 42773
namespace = "USER"
username  = "_SYSTEM"
password  = "SYS"
```

אזהרה: לעולם אל תכניסו קובץ זה ל-source control — הוא מכיל פרטי גישה.
הוסיפו את iris-agentic-dev.toml. ל-gitignore. מיד לאחר יצירתו.

אפשרות ב — IRIS מרוחק דרך משתני סביבה

אם יש לכם מופע IRIS מרוחק, הגדירו את משתני הסביבה האלה במקום קובץ ה-TOML:

```
set IRIS_HOST=your-server.example.com
set IRIS_PORT=52773
set IRIS_NAMESPACE=USER
set IRIS_USERNAME=_SYSTEM
set IRIS_PASSWORD=SYS
```

אפשרות ג — objectscript.conn (תוסף ObjectScript ל-VS Code)

הגדרת `objectscript.conn` מגדירה את **תוסף VS Code ObjectScript** (דפדפן classes, הדגשת תחביר, IntelliSense). היא נפרדת מחיבור ה-MCP. הגדירו אותה לקבלת חוויית הקידוד המלאה:

```
"objectscript.conn": {
    "active":   true,
    "host":     "localhost",
    "port":     42773,
    "ns":       "USER",
    "username": "_SYSTEM",
    "password": "SYS",
    "https":    false
}
```

חלק 3 — אימות החיבור

11. ב-VS Code, עברו ל-**File > Open Folder** ובחרו את תיקיית הפרויקט שלכם (`.iris-agentic-dev.toml` זו שמכילה את קובץ). VS Code מתחבר לשרת ה-MCP באופן אוטומטי.
12. פתחו את לוח הצ'אט של Codex והקלידו:

    ```
    Call check_config and show me the result.
    ```

תגובה מוצלחת נראית כך:

```
{
  "server_version": "1.x.x",
  "namespace": "USER",
  "tools": ["check_config", "iris_execute", "iris_compile", ...]
}
```

13. אם החיבור נכשל, השתמשו במדריך פתרון הבעיות הבא:

תסמין	סיבה סבירה	תיקון
check_config לא נמצא	שרת MCP לא רשום או Codex לא הופעל מחדש	בדקו את נתיב config.toml; הפעילו מחדש את VS Code לאחר עריכתו
"Connection refused"	קונטיינר IRIS Docker לא פועל	הריצו docker ps; אם עצר: docker compose

תסמין	סיבה סבירה	תיקון
על פורט 42773		up -d
"Credentials rejected"	שם משתמש או סיסמה שגויים ב-TOML	אמתו username/_SYSTEM וסיסמה ב-.iris-agentic-dev.toml
Tools רשומים אך iris_execute נכשל	Namespace לא נמצא או למשתמש אין הרשאה	בדקו "namespace = "USER ושלמשתמש IRIS יש %DB_READ/%DB_WRITE
שרת MCP מופעל אך Codex מתעלם ממנו	תוסף Codex דורש הפעלה מחדש לאחר שינוי config.toml	טענו מחדש את חלון VS Code (Ctrl+Shift+P ← "Reload Window")

חלק 4 — Tools לעומת Skills

מה הם Tools?

Tools הם פונקציות נפרדות ש-agent יכול לקרוא — כל אחת מבצעת פעולה קונקרטית אחת על מופע IRIS חי. חשבו על tool כ-פועל.

Tool	מה הוא עושה	פרומפט לדוגמה
`check_config`	ורשימת כל MCP-אימות חיבור ה .הזמינים tools-ה	"Call check_config and show me the result."
`iris_execute`	במסוף ObjectScript הרצת כל ביטוי .IRIS	"Create a ^helloworld global in namespace USER"
`iris_compile`	.והחזרת שגיאות ישירות class הידור	"Compile class Workshop.Person and list any errors."
`iris_query`	והחזרת SQL הרצת שאילתת .תוצאות	"Run SELECT TOP 10 * FROM Workshop.Person"
`iris_admin`	,פעולות ניהול: רשימת משתמשים .namespaces	"List all the users on this IRIS instance"
`iris_production`	productions קריאה ובדיקה של .לאינטרופרביליות	"What's the status of the productions?"
`iris_generate`	.class של stub יצירת שלד או	"Create a class called Workshop.Person"
`iris_unit_test`	הרצת classes של %UnitTest .והחזרת תוצאות עבר/נכשל	"Run all unit tests in package Workshop"

Tool	מה הוא עושה	פרומפט לדוגמה
`iris_doc / docs_introspect`	קריאת מסמכי class וחתימות methods.	"Show me the methods of %Stream.GlobalBinary"

> **טיפ:** ה-agent הוא לא-דטרמיניסטי - הוא עשוי לבחור בין tools שונים לאותה משימה. הנחו אותו על ידי ציון מפורש של ה-tool: "Use iris_admin to list all users."

מה הם Skills?

Skills הם מומחיות ארוזה — קבצי Markdown שמעצבים כיצד ה-agent עובד על פני תהליך עבודה שלם. חשבו על skill כ-ספר הפעלה, לא כפעולה בודדת.

	Tools	Skills
מה זה	פונקציות נפרדות ש-agent יכול לקרוא	מומחיות ארוזה שמעצבת כיצד ה-agent עובד
תפקיד	ביצוע פעולה בודדת וקונקרטית	הנחיית תהליך עבודה שלם
דוגמה	הרצת SQL, קמפול class, שליפת מסמך	תקני קידוד, רשימות Debug שגיאות, תבניות דוחות
חשבו על זה כ	פועל — פעולה	ספר הפעלה — ידע

דוגמה: בקשה מה-agent *"לתקן את שגיאות ה syntax של ObjectScript בחבילה X ולוודא שמתקמפל"* מפעילה את skill ה-`objectscript-repair`, שבתורו קורא ל-`iris_compile` מספר פעמים באופן אוטומטי.

חלק 5 — התקנת Skills

אילו skills זמינים?

ה-repository iris-agentic-dev מגיע עם 34 skills הכוללים ObjectScript, פלטפורמת IRIS, אינטרופרביליות, קונטיינרים ועוד. אלה השימושיים ביותר לסדנת פיתוח:

Skill	מה הוא עושה
`objectscript-review`	רשימת בדיקה של שיטות עבודה מומלצות למניעת טעויות נפוצות

Skill	מה הוא עושה
	ב-ObjectScript.
`objectscript-repair`	תהליך עבודה שלב-אחר-שלב לאבחון ותיקון שגיאות קומפילציה.
`objectscript-debugging`	תהליך מעקב שגיאות: קריאת השגיאה, מעקב שרשרת הקריאות, תיקון הסיבה השורשית.
`objectscript-tdd`	לולאת פיתוח מונחה-בדיקות: כתוב בדיקה — הרץ — תקן — חזור.
`objectscript-unit-test`	כיצד לכתוב ולהריץ UnitTest% של classes.
`ensemble-production`	כיצד לבדוק ולנפות שגיאות ב productions- של Interoperability.
`iris-sql`	תוכניות שאילתה והנחיות אופטימיזציה, SQL דפוסי.
`iris-vector-ai`	מוטמע IRIS ו-AI עבודה עם חיפוש וקטורי של.

העתקת ה-skills למקומם

Skills הם קבצי Markdown. העתיקו אותם אל `~/.claude/skills/` (עבור sessions של Claude Code) או הפנו אליהם דרך הוראות הפרויקט שלכם עבור sessions של Codex.

14. מה-repository המשוכפל, העתיקו את ה-skills:

```
# Windows (PowerShell)
New-Item -ItemType Directory -Force "$env:USERPROFILE\.claude\skills"
Copy-Item iris-agentic-dev\skills\*.md "$env:USERPROFILE\.claude\skills\"
```

15. עבור Codex ב-VS Code: העתיקו את ה-skills הרצויים לסביבת העבודה שלכם והפנו אליהם ב-`AGENTS.md` (ראו חלק 6). Codex קורא הוראות פרויקט מקובץ זה.

16. על ידי שאלה ל-agent אמתו שה-skills יכול לראות את ה:

```
Show me the list of available skills for IRIS development.
```

חלק 6 — הקשר מתמשך עם AGENTS.md

Codex קורא את `AGENTS.md` בתיקיית הפרויקט שלכם בתחילת כל session. כאן אתם אומרים ל-agent את הקונבנציות שלכם, מה אסור, ואילו skills להשתמש. זה שקול לקליטת מפתח פיתוח חדש.

מה לשים ב-AGENTS.md

- קונבנציות שמות: `All classes go in the Workshop package.`

- אזורים אסורים: `Never modify the %SYS namespace.`
- לשימוש Skills: `Apply objectscript-review before saving any ObjectScript class.`
- דוגמאות קנוניות: `See src/Workshop/Person.cls for the class structure to follow.`

יצירת AGENTS.md

17. צרו קובץ בשם `AGENTS.md` בשורש הפרויקט שלכם:

```
# IRIS Development Agent Instructions

## Namespace
Always work in the USER namespace unless explicitly told otherwise.

## Naming conventions
All new classes go in the Workshop package (e.g. Workshop.Person).

## Skills
Run objectscript-review before saving any ObjectScript class.
Use objectscript-repair when diagnosing compile errors.

## Off-limits
Never modify system classes (prefix %).
Never run DROP TABLE or DELETE without explicit confirmation.
```

> **טיפ:** Codex קורא מחדש את AGENTS.md בכל session חדש. עדכנו אותו ככל שהפרויקט מתפתח — זהו מקור האמת היחיד להתנהגות ה-agent בסביבת עבודה זו.

חלק 7 — תרגילים מעשיים

עבדו על התרגילים האלה בסדר. כל אחד מתאמן על tool שונה ובונה על הקודם.

תרגיל 1 — check_config: אימות ההגדרה

18. בצ'אט Codex, הקלידו:

```
Call check_config and show me the result.
```

אמתו שאתם רואים: גרסת שרת, namespace, ורשימה של כל ה-tools הזמינים.

תרגיל 2 — iris_execute: יצירה וקריאה של Global

19. צרו global:

```
Create a ^helloworld global in namespace USER
```

20. קראו אותו חזרה:

```
Read the value of ^helloworld from namespace USER
```

תרגיל 3 — iris_admin: רשימת משתמשים

21. נסו את שני הפרומפטים — שימו לב ש-agent עשוי לבחור tools שונים:

```
List all the users on this IRIS instance
Use iris_admin to list all users
```

תרגיל 4 — iris_production: בדיקת סטטוס

22. אם למופע IRIS שלכם יש productions של Interoperability:

```
What's the status of the productions?
```

23. נסו מחדש, אבל הפעם בעברית:

```
מה הסטטוס של ה־production?
```

תרגיל 5 — iris_generate: יצירת Class

24. בקשו מה-agent ליצור persistent class:

```
Create a persistent class Workshop.Person with properties Name (String)
and PersonID (Integer). Save it under src/ using our naming conventions.
```

25. צפו בו מקמפל ומתקן בעצמו. לאחר מכן אמתו:

```
Read back the source of class Workshop.Person
```

או פשוט פתחו את Workshop.Person והסתכלו בתוכן שלו.

תרגיל 6 — docs_introspect: חקירת ה-API

26. חקרו class IRIS קיים:

```
Show me the methods of class %Stream.GlobalBinary
```

תרגיל 7 — iris_compile: תיקון Class שבור

תרגיל זה מדגים את לולאת התיקון העצמי האג'נטי — ה-agent מקמפל, קורא את השגיאה ומתקן אוטומטית.

27. בקשו מה-agent ליצור class עם שגיאה מכוונת ולאחר מכן לקמפל אותו:

```
Create a class Workshop.BrokenClass with an intentional ObjectScript
syntax error. Then compile it and fix all compile errors automatically.
```

28. צפו ב-agent מבצע איטרציות: כתיבה — קמפול — קריאת שגיאה — תיקון — קמפול מחדש.

תרגיל 8 — iris_query: הרצת SQL

29. שאלו את ה-class שיצרתם בתרגיל 5:

```
Run SELECT TOP 10 ID, Name FROM Workshop.Person ORDER BY Name
```

30. הרחיבו את השאילתה עם פילטר:

```
Now add a WHERE clause that filters rows where Name starts with 'A'
```

תרגיל 9 — iris_unit_test (אופציונלי): כתיבה והרצה של בדיקות

31. בקשו מה-agent לכתוב ולהריץ בדיקת יחידה עבור Workshop.Person:

```
Write a %UnitTest class for Workshop.Person that verifies a new instance
can be saved and retrieved. Run it and fix any failures.
```

> **טיפ:** תרגיל זה מממש את שלב הבדיקה המלא של לולאת הפיתוח ב-IRIS ומראה כיצד tools (iris_compile, iris_unit_test) ו-skills (objectscript-tdd) משתלבים.

חלק 8 — דפוסי עבודה מעשיים

פרומפטים שעובדים

- ציינו מטרה + מגבלות + קריטריוני קבלה: *"Create class X such that it compiles and unit tests pass."*
- בקשו הסבר לפני עריכה: *"Before making any changes, describe what you plan to do."*
- העדיפו שלבים קטנים וניתנים לאימות על פני שכתובים גדולים.

תהליכי עבודה בטוחים

- התחילו על מופע Docker מקומי. מחקו ובנו מחדש אם משהו משתבש.
- השתמשו במשתמש agent — לא SuperUser או חשבון `_SYSTEM` ייעודי עם הרשאות מינימום.
- התחילו עם tools לקריאה בלבד (`iris_query`, `iris_doc`, `check_config`). פתחו tools לכתיבה בהדרגה.
- השתמשו בקובץ `AGENTS.md` לציון גבולות namespace ומה ש-agent לעולם לא יגע בו.

חלק 9 — שיטות עבודה מומלצות לאבטחה

שליטה בגישה ברמת IRIS

- צרו חשבון IRIS ייעודי (לדוגמה, AgentDev) עם תפקידים מינימליים נדרשים: `%DB_READ` ו-`%DB_WRITE` על namespace הפיתוח בלבד.
- לעולם אל תשתמשו ב-`_SYSTEM` או בתפקיד SuperUser מחוץ לקונטיינר מקומי זמני.
- ה-binary של MCP מתחבר דרך IRIS Atelier REST API הסטנדרטי. כל בקרת הגישה חייבת להיאכף ברמת משתמש/תפקיד IRIS.

הגבלת הרשאות ה-Agent

- ב-`AGENTS.md`, ציינו גבולות namespace ומה ש-agent לעולם לא יגע בו.
- ניסיוני ייעודי בסביבת הפיתוח, לעולם לא מול נתוני ייצור או namespace רק מול שקלו להריץ את ה-agent משותפים.

אזהרה: אל תכניסו פרטי גישה ל-source control.
השתמשו במשתני סביבה או מנהל סודות.
קובץ iris-agentic-dev.toml. הוא מקומי למחשב שלכם — הוסיפו אותו ל-.gitignore.

עזר מהיר — פרומפטים שימושיים

פרומפט	מטרה
Call check_config and show me the result.	אימות חיבור
Create a ^helloworld global in namespace USER	יצירת global
Read the value of ^helloworld from namespace USER	קריאת global
List all the users on this IRIS instance	רשימת משתמשים
What's the status of the productions?	בדיקת productions
Create a persistent class Workshop.Person with Name (String) and PersonID (Integer)	יצירת class
Compile class Workshop.Person and list any errors	הידור class
Fix all compile errors in class Workshop.Person	תיקון שגיאות קומפילציה
Run SELECT TOP 10 * FROM Workshop.Person	שאילתת SQL
Show me the methods of class %Stream.GlobalBinary	חקירת API של class
Write a %UnitTest class for Workshop.Person and run it	כתיבת בדיקות יחידה
Do you have any skills available for IRIS development?	רשימת skills
re-open workspace (Codex)	רענון הקשר
Fix the ObjectScript errors in package Workshop so it compiles and tests pass	תיקון ObjectScript